

Proofs for the ZAP Zero-Copy Throughput Paper

ZAP Protocol Authors
zap-protocol.io

2026

Abstract

This document gives full proofs of the two theorems stated in the companion paper, plus the supporting lemmas. We work in a unit-cost RAM model augmented with a two-level I/O budget that distinguishes cache-resident from main-memory accesses. We assume schemas are known to both endpoints at compile time, that pointers are 8-byte aligned, and that the network presents bytes already deposited into a contiguous arena (the standard `recvmsg` plus `mmap`-friendly pattern).

1 Model

Definition 1 (Unit-cost RAM with cache budget). We use a RAM machine with word size $w = 64$. Each instruction (arithmetic, branch, register-to-register move) costs 1 unit. Memory accesses cost c_h for L1-resident data and c_m for main memory, with $c_m \gg c_h$. Cache lines are 64 bytes. Any byte range of length ℓ touched once costs at most $\lceil \ell/64 \rceil$ cold lines plus $O(\ell/w)$ instructions.

Definition 2 (Arena allocator). An arena is a contiguous byte range with a bump pointer. Allocation of an aligned k -byte object costs $O(1)$: one compare against the arena end, one add to the bump pointer, one alignment mask.

Definition 3 (Wire formats). We consider four wire formats:

- **ZAP**: arena of segments; structs are positional; pointers are relative.
- **Protobuf (PB)**: tag-length-value records; varint tags and lengths.
- **JSON**: textual key-value pairs; numbers are decimal strings.
- **FlatBuffers (FB)**: positional like ZAP but with a vtable indirection per struct (included for context).

Definition 4 (Workload parameters). A request has f logical fields, total wire size n bytes, and p of those bytes are variable-length payload (strings, blobs, lists) that the sender chose to allocate inside the request arena.

2 Constant-time arena allocation

Lemma 5 (Arena allocation is $O(1)$). *Allocating an aligned struct of size $k \leq 2^{16}$ in an arena takes $O(1)$ instructions and $O(1)$ cache lines amortized.*

Proof. The bump pointer b and the arena end e both fit in registers. The allocation is

$$b' = \text{align}_8(b + k); \quad \text{if } b' > e \text{ go to overflow path; } \quad b \leftarrow b'.$$

Three arithmetic operations and one branch: $O(1)$ instructions. The cache cost is incurred only on the first byte of the next cache line, so amortized over $64/k$ allocations it is $O(1/k)$ lines. The overflow path falls through to a fresh segment allocation, which is itself $O(1)$ (a single `mmap`-style call to a free list). $\square$

Lemma 6 (Field access is $O(1)$ in field count). *For a ZAP struct, reading or writing field f_k takes $O(1)$ instructions independent of the total number of fields f .*

Proof. Compile-time generated accessors compute the offset $\text{off}(f_k)$ as a constant. The access is then a single load from $\text{base} + \text{off}(f_k)$ (one displacement-mode memory operation on x86_64). No tag decoding, no table lookup, no indirection. $\square$

Remark. This is the property that distinguishes ZAP from FlatBuffers, which performs a vtable lookup adding a load and an addition per field access. The asymptotic class is the same ($O(1)$) but the constant is roughly $2\times$ smaller for ZAP.

3 Per-format serialization cost

Lemma 7 (ZAP serialization cost). *Constructing a ZAP message containing f fixed-layout fields and p bytes of variable payload costs $L_{\text{ser}}^{\text{ZAP}}(f, p) = O(f) + \Theta(p)$ in instructions, and the $O(f)$ term is a sum of independent stores that the writer issues anyway.*

Proof. By Theorem 5 the arena is initialized in $O(1)$. By Theorem 6 each fixed field is one store. The writer must store each field at least once, so this is the same cost as “writing the answer”—there is no *additional* marshaling work. Variable-length payload is copied byte-by-byte at $\Theta(p)$. The amortized cost over a request that already produces these bytes is therefore $L_{\text{ser}}^{\text{ZAP}}(f, p) = \Theta(p) + O(1)$ beyond the unavoidable field stores. $\square$

Lemma 8 (Protobuf serialization cost). *Encoding a Protobuf message with f fields and p payload bytes costs $L_{\text{ser}}^{\text{PB}}(f, p) = \Omega(f) + \Theta(p)$, and the $\Omega(f)$ term cannot be eliminated by a positional representation because the wire format itself requires per-field tag bytes.*

Proof. Each present field f_k contributes at least one varint tag byte plus, for length-delimited types, a varint length prefix. Computing the varint requires a loop bounded by 5 bytes for 32-bit tags but constant for any specific tag; nevertheless it is ≥ 1 store *per field*, even when the field value is otherwise free. Therefore the wire-format definition forces $\Omega(f)$ stores, and no encoder can do fewer. $\square$

Lemma 9 (JSON serialization cost). *Canonical JSON encoding of f fields with p payload bytes costs $L_{\text{ser}}^{\text{JSON}}(f, p) = \Theta(n + f \log f)$ where n is the total output size.*

Proof. Each field contributes its key as text, a colon, the value, and a comma; this is $\Theta(n)$ in output bytes. Canonical JSON requires keys to be sorted; sorting f keys is $\Theta(f \log f)$. Number formatting (double-to-decimal-string with round-trip guarantee) is $\Theta(1)$ per number but with a much larger constant than a memory store. $\square$

4 Throughput Theorem

Theorem 10 (Throughput bound for zero-copy wire). *Consider a workload offering requests of size n bytes at offered load λ , with link bandwidth B and per-request fixed CPU cost σ (framing, dispatch, scheduler entry). The achievable steady-state throughput T for a wire format with serialization cost $L_{ser}(n, f)$, deserialization cost $L_{de}(n, f)$, and per-message header overhead h satisfies*

$$T \leq \min\left(\frac{B}{n+h}, \frac{1}{\sigma + L_{ser}(n, f) + L_{de}(n, f)}\right). \quad (1)$$

For ZAP under fixed-layout structs, $L_{ser}(n, f) = L_{de}(n, f) = O(1)$ and $h = O(w)$, so as $n \rightarrow \infty$, $T \rightarrow B/n$. For Protobuf-over-HTTP/2 with HPACK, $h \geq h_{\text{HPACK}} + h_{\text{frame}}$ and the CPU bound binds first at small n , so $T \leq r \cdot B/n$ for some $r \in [0.6, 0.7]$ on standard hardware.

Proof. The link cannot deliver more than B bytes per second; therefore at most $B/(n+h)$ messages per second can cross the wire. This is the first term of the min.

The CPU pipeline must (i) pay the per-request fixed cost σ , (ii) serialize, (iii) deserialize. By Little’s law on a single core, the steady-state rate cannot exceed $1/(\sigma + L_{ser} + L_{de})$. This is the second term.

For ZAP, by Theorem 7 and the symmetric statement for deserialization (which is the identity function on the byte buffer plus the constant-cost root pointer chase), $L_{ser} + L_{de} = O(1)$. The header overhead is the segment table plus root pointer, which is $O(w)$ words. Therefore the second term is $\Theta(1/\sigma)$, independent of n , and the first term dominates as $n \rightarrow \infty$.

For Protobuf-over-HTTP/2, by Theorem 8, $L_{ser}(n, f) + L_{de}(n, f) = \Omega(f) + \Theta(n)$. HPACK header decoding contributes a per-message constant σ_{HPACK} that is non-negligible (measured in published benchmarks at 0.5–1.5 μs). The CPU bound is therefore $1/(\sigma + \sigma_{\text{HPACK}} + \Omega(f))$, which on commodity hardware empirically binds at 0.6–0.7 of the link bound for $n \in [128, 8192]$ bytes. Hence $T \leq r \cdot B/n$ with r in that range. $\square$

Corollary 11. *The crossover point at which ZAP saturates the link, while Protobuf remains CPU-bound, is approximately $n^* = (\sigma_{\text{HPACK}} + \alpha f) \cdot B/(1 - r)$, where α is the per-field cost. For $\sigma_{\text{HPACK}} = 1 \mu\text{s}$, $\alpha = 10 \text{ ns}$, $f = 32$, $B = 10 \text{ Gbps}$, $r = 0.65$, we get $n^* \approx 3.7 \text{ KiB}$.*

5 Tail-Latency Independence Theorem

Theorem 12 (Tail-latency independence from field count). *Let $L^{(q)}(f, p)$ denote the q -th quantile of single-RPC latency under i.i.d. payloads with f fields and bounded payload bytes p . For ZAP,*

$$L_{\text{ZAP}}^{(q)}(f, p) = L_{\text{ZAP}}^{(q)}(p) + O(1),$$

i.e. the q -th quantile is asymptotically independent of f . For Protobuf and JSON,

$$L_{\text{PB}}^{(q)}(f, p) = \Omega(f) + L_{\text{PB}}^{(q)}(p),$$

and the slope at $q = 0.999$ exceeds the slope at $q = 0.5$ by a multiplicative factor that grows with allocator and HPACK dynamic-table state.

Proof. By Theorem 7, ZAP serialization cost is $O(1)$ in field count plus $\Theta(p)$ in payload bytes. The same holds for deserialization (identity map). The variance contribution to any quantile is therefore bounded by the variance of the constant-time fixed cost, which is $O(1)$ regardless of f .

For Protobuf, by Theorem 8, each field contributes at least one tag-byte store and, for length-delimited fields, one varint length-prefix store. These stores can each miss the L1 cache, and the probability that at least one of f such stores misses grows as $1 - (1 - m)^f \approx mf$ for small per-store miss probability m . Therefore the expected number of L1 misses scales with f , and at high quantiles the realized number of misses scales faster than at the median due to the heavy-tailed distribution of cache-miss penalties.

For JSON, the canonical-key sort is $\Theta(f \log f)$ in the median case, and adversarial input can push it toward $\Theta(f^2)$ in quicksort-based implementations, dominating high quantiles.

For the HPACK component, dynamic-table eviction during congestion adds tail-only latency proportional to header set cardinality, which correlates with f when each field becomes a header (gRPC trailers, metadata, etc.). $\square$

Corollary 13. *The $p999/p50$ ratio for ZAP is bounded by the noise floor of the network and scheduler, typically $3\text{--}5\times$. For Protobuf-over-HTTP/2 under congestion, the same ratio observed in fleet measurements exceeds $20\text{--}50\times$.*

6 Discussion of Lower Bounds

Remark. The throughput bound Equation (1) is tight in the sense that ZAP achieves the $B/(n + h)$ term up to a small constant in practice. The Protobuf bound is also tight: there is no encoder that avoids the per-field tag byte without changing the wire format itself, and changing the wire format yields a different protocol (which is what ZAP and FlatBuffers are).

Remark. The tail bound Theorem 12 relies on the i.i.d. assumption. Adversarial workloads can be constructed to make ZAP’s tail worse, e.g. by forcing arena overflow at every request to trigger segment allocation. The expected behavior under realistic load remains as stated.

7 Summary

We have proved that ZAP’s serialization and deserialization costs are constant in field count, that the resulting throughput approaches link rate as message size grows, and that tail latency is asymptotically independent of field count. These bounds match the predicted speedups in the companion paper. The proofs make explicit their model assumptions; relaxing them (e.g. allowing adversarial arena overflow) changes constants but not asymptotics.